1. SELECT: choose what you want to see

Start here. Read one idea at a time, then move to practice on page 15.

What this does

A SQL table is like a spreadsheet: columns describe things, and rows contain records. **SELECT** chooses the columns to display. **FROM** names the table. **WHERE** keeps only rows that meet a condition.

Input: orders

order_id	customer_id	amount
101	1	100
102	1	50
103	2	80

Example: show orders worth at least 80

```
SELECT order_id, amount
FROM orders
WHERE amount >= 80
ORDER BY amount DESC;
```

Exact output

order_id	amount
101	100
103	80

Order 102 disappears because 50 is below 80. customer_id disappears because we did not select that column. **ORDER BY amount DESC** puts the largest amount first; ASC means smallest first.

A few essentials

IN ('East', 'West') means East OR West; NOT IN excludes both, but also excludes NULL regions. Text uses single quotes: status = 'completed'. Numbers need no quotes. AND requires both conditions; OR requires at least one. **LIMIT 5** keeps at most five result rows.

> **Think:** Which table has the information? Which rows qualify? Which columns should the answer show?

MY NOTES / QUESTIONS TO REVISIT

2. JOIN: connect related tables

Use a join when the information you need is spread across two tables.

What this does

A customer table may hold names while an order table holds purchases. A join connects them using a shared key. **ON** describes the match: the order's customer_id must equal the customer's customer_id.

Input: customers (left) and orders (right)

customer_id	name
1	Asha
2	Ben
3	Cara

order_id	customer_id
101	1
102	1
103	2

Example: keep every customer, even without an order

```
SELECT c.name, o.order_id
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id
ORDER BY c.customer_id, o.order_id;
```

Exact output

name	order_id
Asha	101
Asha	102
Ben	103
Cara	NULL

Asha appears twice because she has two orders. Cara stays because **LEFT JOIN** preserves every row from the left table. NULL means no matching order here. An ordinary **JOIN** (INNER JOIN) would drop Cara.

AS c and **AS o** are short table names. ON c.customer_id = o.customer_id works the same as reversing the two sides of this equality.

> **Watch out:** joins can repeat rows. If you add up a customer-level value after joining to many orders, you may count it more than once.

MY NOTES / QUESTIONS TO REVISIT

3. Join edge case: ON versus WHERE

The same condition can change the answer depending on where you put it.

Input: customers (left) and orders (right)

customer_id
1
2
3

order_id	customer_id	status
101	1	completed
102	2	cancelled

A. Match only completed orders; keep all customers

```
SELECT c.customer_id, o.order_id
FROM customers AS c
LEFT JOIN orders AS o
  ON o.customer_id = c.customer_id
 AND o.status = 'completed'
ORDER BY c.customer_id;
```

customer_id	order_id
1	101
2	NULL
3	NULL

ON determines which orders match. Customer 2 has an order, but no completed match; customer 3 has no order at all. LEFT JOIN preserves both, with NULL right-hand columns.

B. Instead, put the status test in WHERE

```
SELECT c.customer_id, o.order_id
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
WHERE o.status = 'completed'
ORDER BY c.customer_id;
```

customer_id	order_id
1	101

> **WHERE** runs after the join. It removes customer 2 because cancelled is not completed, and customer 3 because NULL = 'completed' is UNKNOWN. Adding OR o.order_id IS NULL still would not rescue customer 2: a cancelled row matched.

MY NOTES / QUESTIONS TO REVISIT

4. NULL: missing is not zero

SQL conditions can be TRUE, FALSE or UNKNOWN. WHERE keeps only TRUE.

Input: customers

customer_id	region	score
1	East	10
2	West	NULL
3	NULL	20

Example: exclude East but keep unknown regions

```
SELECT customer_id, region
FROM customers
WHERE region <> 'East' OR region IS NULL
ORDER BY customer_id;
```

customer_id	region
2	West
3	NULL

region <> 'East' is TRUE for West but UNKNOWN for NULL. **OR region IS NULL** explicitly includes the unknown region. Writing region = NULL would not test for missing values; use IS NULL or IS NOT NULL.

Aggregates ignore missing values, except COUNT(*)

```
SELECT COUNT(*) AS n_rows, COUNT(score) AS n_scores,
       SUM(score) AS total, AVG(score) AS avg_score
FROM customers;
```

n_rows	n_scores	total	avg_score
3	2	30	15

COUNT(*) sees three rows; **COUNT(score)** sees two known scores. AVG divides 30 by 2, not 3. AVG(COALESCE(score,0)) would give 10 because it assumes the missing score is actually zero.

> **Helpers:** COALESCE(x,0) returns 0 only when x is NULL. NULLIF(x,0) returns NULL when x is 0, useful for safe division. NULL is different from 0 and the empty string ''.

Other edges: NULL + 5 stays NULL. NULL = NULL is UNKNOWN; normal equality joins do not match two NULL keys. GROUP BY does put NULL grouping values into one group. SUM/AVG on empty ungrouped input return NULL; COUNT returns 0.

MY NOTES / QUESTIONS TO REVISIT

5. GROUP BY: summarize several rows

Use grouping for questions such as how many, how much, or what is the average per customer.

What this does

GROUP BY puts rows with the same key into a group. An aggregate then summarizes each group: COUNT counts, SUM adds, AVG averages, and MIN/MAX find the smallest/largest value.

Input: orders

order_id	customer_id	amount
101	1	100
102	1	50
103	2	80

Example: count orders and total spend per customer

```
SELECT customer_id, COUNT(*) AS n_orders,
       SUM(amount) AS total_spend
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

Exact output

customer_id	n_orders	total_spend
1	2	150
2	1	80

The two rows for customer 1 become one summary row. **AS n_orders** names the count column; **AS total_spend** names the sum column. Those aliases are the output headers.

WHERE versus HAVING

WHERE filters the original rows before grouping. **HAVING** filters the summaries afterward. To keep only customers with at least two orders, add HAVING COUNT(*) >= 2 before ORDER BY; only customer 1 would remain.

> **Count carefully:** COUNT(*) counts rows. COUNT(o.order_id) counts non-null order IDs. After a LEFT JOIN, the second choice gives a customer with no orders a count of 0, rather than counting their placeholder row as 1.

MY NOTES / QUESTIONS TO REVISIT

6. CASE: calculate a conditional metric

Use CASE when different rows should contribute differently to the same calculation.

What this does

CASE WHEN ... THEN ... ELSE ... END is SQL's if/else expression. For a success rate, turn each success into 1 and each failure into 0, then add the successes and divide by all eligible rows.

Input: orders

order_id	status
101	completed
102	completed
103	cancelled
104	cancelled

Example: percentage of completed orders

```
SELECT ROUND(
  100.0 * SUM(CASE WHEN status = 'completed'
                   THEN 1 ELSE 0 END) / COUNT(*),
  2) AS completed_pct
FROM orders;
```

Exact output

completed_pct
50.00

CASE produces 1, 1, 0, 0. SUM gives 2; COUNT(*) gives 4. So 100 × 2/4 = 50%. ROUND(..., 2) requests two decimal places.

Two helpers worth knowing

NULL means missing or unknown, not zero. Test it with IS NULL. **COALESCE(x, 0)** uses zero if x is NULL. **NULLIF(denominator, 0)** makes a zero denominator NULL so an undefined rate stays NULL instead of dividing by zero.

> **Pick the denominator first.** For a combined rate, add the successes and opportunities, then divide. Do not simply average group percentages when the group sizes differ.

MY NOTES / QUESTIONS TO REVISIT

7. Subqueries: compute or check first

A smaller query can provide one value, a list, or the answer to “does a match exist?”

Input: employees (left) / exact output (right)

employee_id	salary
1	50
2	70
3	90

employee_id	salary
3	90

```
SELECT employee_id, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY employee_id;
```

Inner SELECT: calculates the overall average, 70. **WHERE:** compares each salary with 70. **Outer SELECT:** shows only the qualifying employee's ID and salary. A scalar subquery must supply at most one row and one column.

Missing matches: NOT EXISTS

Now suppose customers has IDs 1,2,3, while orders.customer_id contains 1 and NULL. The NULL order key represents missing customer information.

```
SELECT c.customer_id
FROM customers AS c
WHERE NOT EXISTS (
  SELECT 1 FROM orders AS o
  WHERE o.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

customer_id
2
3

Inner WHERE: looks for an order belonging to the current customer. **NOT EXISTS:** keeps the customer only if there is no match. SELECT 1 is just a placeholder; it does not add a column to the result.

> **NOT IN trap:** customer_id NOT IN (SELECT customer_id FROM orders) returns no rows on these inputs. IDs 2 and 3 have no equal value, but the NULL makes the condition UNKNOWN. NOT EXISTS avoids that particular trap.

MY NOTES / QUESTIONS TO REVISIT

8. Windows: compare rows within a group

Use a window when each original row should remain visible alongside a rank or comparison.

What this does

GROUP BY collapses rows into summaries. A **window function** keeps the rows and calculates something across related rows. PARTITION BY defines the group; ORDER BY inside OVER defines the comparison order.

Input: employees (left) / exact output (right)

employee_id	department	salary
1	Eng	100
2	Eng	100
3	Eng	90
4	Sales	80

employee_id	salary	pay_rank
1	100	1
2	100	1
3	90	2
4	80	1

Example: rank distinct salaries within each department

```
SELECT employee_id, salary,
       DENSE_RANK() OVER (
         PARTITION BY department ORDER BY salary DESC
       ) AS pay_rank
FROM employees
ORDER BY employee_id;
```

Employees 1 and 2 tie at rank 1. The next distinct salary gets rank 2. Sales starts again at rank 1 because it is a separate partition. The output includes pay_rank because the SELECT explicitly requests it.

Which function should I use?

DENSE_RANK: ties share a rank without gaps. **RANK:** ties share a rank but leave gaps: 1,1,3. **ROW_NUMBER:** assigns different numbers: 1,2,3; add a unique tie-breaker when a specific row must win.

LAG(value) reads the previous row in window order. Useful for comparing with a prior observation, but the prior row is not necessarily the previous calendar day.

Running calculations: SUM(x) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) adds values from the first row through the current one. ROWS BETWEEN 6 PRECEDING AND CURRENT ROW uses at most seven rows; early rows have shorter windows.

> **Top N per group:** put the ranking query in a CTE, then filter its rank in an outer WHERE. MySQL cannot filter a window alias in WHERE in the same SELECT.

MY NOTES / QUESTIONS TO REVISIT

9. Your full query: each clause explained

Goal: up to five East customers with at least two completed orders. Inputs and row trace follow on page 10.

```
SELECT c.customer_id, COUNT(o.order_id) AS n_orders
FROM customers AS c
LEFT JOIN orders AS o
  ON o.customer_id = c.customer_id
 AND o.status = 'completed'
WHERE c.region = 'East'
GROUP BY c.customer_id
HAVING n_orders >= 2
ORDER BY n_orders DESC, c.customer_id
LIMIT 5;
```

Read it one piece at a time

SELECT c.customer_id: display the customer ID. **COUNT(o.order_id):** count matched, non-null order IDs. **AS n_orders:** name that output column n_orders.

FROM customers AS c: start with customers and nickname the table c. **LEFT JOIN orders AS o:** attach matching orders, nickname them o, and initially keep customers with no matches.

ON o.customer_id = c.customer_id: an order belongs to the customer whose ID matches. Reversing the two sides of this equality changes nothing. **AND o.status = 'completed':** only completed orders qualify as matches.

WHERE c.region = 'East': keep East customer rows. **GROUP BY c.customer_id:** collect each customer's rows so COUNT produces one count per customer.

HAVING n_orders >= 2: keep groups with two or more completed orders. MySQL permits this SELECT alias in HAVING. WHERE n_orders >= 2 would not work here.

ORDER BY n_orders DESC, c.customer_id: largest counts first; tied counts use customer ID ascending. **LIMIT 5:** take at most five rows after sorting; it does not force five rows or include every tie at the cutoff.

> The output has exactly two columns: **customer_id** and **n_orders**. c and o are table aliases; n_orders is an output alias. Internal join columns do not automatically appear.

MY NOTES / QUESTIONS TO REVISIT

10. How the query changes the rows

This is a conceptual trace of page 9. The optimizer can use a different physical execution plan.

Input: customers (left) and orders (right)

customer_id	region
1	East
2	East
3	West
4	East

order_id	customer_id	status
101	1	completed
102	1	completed
103	1	cancelled
104	2	completed
105	3	completed

1. LEFT JOIN / ON: attach completed matches

c.customer_id	c.region	o.order_id
1	East	101
1	East	102
2	East	104
3	West	105
4	East	NULL

Cancelled order 103 is not a match. Customer 4 still appears once, with NULL on the order side.

2. WHERE: remove West. GROUP BY: summarize East

customer_id	COUNT(o.order_id)
1	2
2	1
4	0

Customer 3 is removed by WHERE. Customer 4 has zero matched order IDs, although its placeholder means COUNT(*) would be 1.

3. HAVING, SELECT, ORDER BY, LIMIT: exact output

customer_id	n_orders
1	2

> HAVING removes counts below 2; SELECT names the count n_orders; sorting and LIMIT leave this one row. Because HAVING removes all zero-match customers, an INNER JOIN gives the same final answer for this particular query.

MY NOTES / QUESTIONS TO REVISIT

11. Edge cases to check before practice

Use this page as a short debugging checklist, not as a list of rules to memorize without context.

Joins: which rows survive and how many?

One-to-many: one customer with three orders becomes three joined rows. **Many-to-many:** two matches on one side and three on the other can create six pairs. Join on the intended keys; equal names or equal amounts are not necessarily the same entity.

Inflated totals: an order worth 100 joined to three items repeats 100 three times. Aggregate item rows to one row per order before joining, or calculate the metric directly at its proper grain. SUM(DISTINCT amount) is not a repair: it also merges different orders with identical amounts.

No matches: LEFT JOIN preserves the left row. To find missing right rows, test a non-nullable right key, e.g. WHERE o.order_id IS NULL. Testing a nullable business column can mistake a real matched row for an absent one.

Self-join: use two aliases to represent roles in the same table, such as employee and manager. A missing manager needs LEFT JOIN if the employee must remain. RIGHT JOIN reverses which side is preserved. CROSS JOIN produces all pairs. MySQL has no native FULL OUTER JOIN.

Grouping, windows and output

GROUP BY produces one row per group; a window keeps individual rows. Empty input with GROUP BY gives no groups; COUNT(*) without GROUP BY returns one row with 0. Use a stable tie-breaker for ROW_NUMBER or LIMIT; avoid adding an ID to a salary rank when salary ties should remain tied.

LAG reads the prior observed row, not necessarily yesterday. A seven-row rolling window spans seven calendar days only if there is one row for every day. NULL inputs, duplicate timestamps and missing dates need deliberate treatment.

MySQL reminders

Use a half-open timestamp range: ts >= '2026-10-01' AND ts < '2026-11-01'. DATE(ts) extracts the date; DATE_ADD(d, INTERVAL 1 DAY) adds a day. A primary key uniquely identifies a row; a foreign key enforces a reference; an index can speed lookups at a storage/write cost.

> **Five checks:** intended grain, correct matching keys, correct denominator, explicit NULL/tie handling, and exact selected columns/aliases. Test a tiny example with no match, multiple matches and missing data.

MY NOTES / QUESTIONS TO REVISIT

12. Statistics: summarize and reason

Focus on what a number means before choosing its formula.

Average and spread

The **mean** is sum/count. The **median** is the middle sorted value; it is less affected by outliers. The **mode** is the most frequent value. Standard deviation (SD) measures spread in the original units; variance is SD squared.

For 2, 3, 3, 4, 18: mean = 6, median = 3, mode = 3. The large value pulls the mean up. If groups differ in size, use a weighted mean: 20 people averaging 80 plus 30 averaging 90 gives (20×80 + 30×90)/50 = 86.

Probability: define the group you are counting

A complement is what does not happen: P(not A) = 1 - P(A). For independent events, multiply probabilities for both occurring. For at least one success, it is often easier to find 1 - P(no successes). Three independent 20% chances give $1 - 0.8^3 = 48.8\%$.

Conditional probability means probability within a known group. P(A given B) = P(A and B)/P(B). It is not generally the same as P(B given A). A table helps keep the denominator straight.

Example: qualified leads and a screening rule

Lead type	Flagged	Not flagged	Total
Qualified	80	20	100
Not qualified	180	720	900
Total	260	740	1,000

Among flagged leads, qualified share = 80/260 = **30.77%**. Among qualified leads, flagged share = 80/100 = **80%**. These reverse the condition and therefore use different denominators.

> **Recognize distributions:** binomial counts successes in independent fixed-probability trials; normal is the familiar bell curve. For normal data, roughly 95% of observations lie within two SD of the mean.

MY NOTES / QUESTIONS TO REVISIT

13. Statistics: uncertainty and evidence

Know the difference between spread, uncertainty and evidence against a claim.

SD versus standard error

SD: how much individual observations vary. **Standard error (SE):** how much an estimate would vary across repeated samples. For an independent sample mean, estimate SE as s/sqrt(n). With SD 20 and n=100, SE is 2. Four times the sample size halves SE.

Confidence intervals

A confidence interval is an estimate plus/minus a margin for uncertainty. For a mean with known population SD, a 95% normal interval is mean ± 1.96 × SE. Mean 50, known SD 10 and n=100 gives SE=1 and interval [48.04,51.96].

Across repeated samples, a valid 95% interval procedure covers the fixed population parameter about 95% of the time. It does not mean 95% of individual observations lie inside that interval. When population SD is estimated, use a t method under its assumptions.

Hypothesis tests

A test starts with a null claim H0. A **p-value** asks how surprising the data would be if H0 and the statistical model held. If p ≤ the chosen significance level, reject H0; otherwise fail to reject. p=0.03 does not mean H0 has a 3% chance of being true.

Type I error: reject a true H0. **Type II error:** fail to reject a false H0. For mean comparisons, recognize one-sample t, independent-group Welch t, paired t for the same people before/after, and ANOVA for three or more groups.

Relationships and experiments

Correlation measures linear association; it does not establish cause. **Regression slope** describes predicted change in the outcome per unit of a predictor. Random assignment supports causal comparisons; random sampling supports population generalization. A large sample does not fix selection bias.

> **Interpret results:** statistical significance does not tell you whether an effect is large enough to matter in business. Check the effect size and uncertainty too.

MY NOTES / QUESTIONS TO REVISIT

14. Data interpretation: choose the base

Most mistakes come from using the wrong denominator or ignoring units.

Percentages: three common questions

Share: part/whole. **Growth:** (new-old)/old. **Percentage-point change:** subtract two percentages. Moving from 20% to 25% is +5 percentage points, but +25% relative growth because 5/20 = 25%.

Example table: revenue and profit in $000

Region	Old revenue	New revenue	New profit
North	120	150	30
South	80	100	15
Total	200	250	45

Overall growth: (250-200)/200 = 25%.
North share of new revenue: 150/250 = 60%.
Overall profit margin: 45/250 = 18%.
North contribution to the increase: (150-120)/(250-200) = 60%.

Weighted rates and successive changes

If one group has 20 conversions from 100 visitors and another has 90 from 900, combine counts: 110/1000 = **11%**. Averaging the two rates would wrongly give 15%.

A 20% rise followed by a 20% fall gives 1.2 × 0.8 = 0.96, a **4% decrease**. Multiply growth factors rather than adding changes. Revenue = volume × average price, so volume +20% and price -10% gives revenue +8%.

Read charts carefully

Check units, time periods, labels and the axis scale. Bars compare categories; lines show trends; scatterplots show relationships. A truncated axis can exaggerate differences. Only claim exact values if the labels or scale support them.

> **Always ask:** a percentage of what? dollars or thousands of dollars? total or per person? Compare like with like, estimate the answer first, and round at the end.

MY NOTES / QUESTIONS TO REVISIT

15. Then practice: short LeetCode list

Use MySQL 8+ and Python 3. Start after the review; check off problems you can solve independently.

MySQL: six targeted problems

[ ] Customer Who Visited but Did Not Make Any Transactions
Practice matching tables and finding missing orders.
[ ] Managers with at Least 5 Direct Reports
Practice grouping and filtering counts with HAVING.
[ ] Average Selling Price
Practice weighted averages and date-range matches.
[ ] Monthly Transactions I
Practice CASE, conditional sums and monthly groups.
[ ] Confirmation Rate
Practice a rate with missing matches.
[ ] Department Top Three Salaries
Practice ranking within groups and keeping ties.

Python: four targeted problems

[ ] Contains Duplicate - sets and membership.
[ ] Valid Anagram - count character frequencies.
[ ] Two Sum - dictionary lookup and indices.
[ ] Best Time to Buy and Sell Stock - track the best answer in one pass.

Statistics and interpretation check

Without looking back, explain the two different percentages in the lead table on page 12, calculate the confidence interval on page 13, and redo the revenue/rate examples on page 14. Name each denominator.

Optional MySQL references: Joins | NULL behavior | Aliases.

> **Practice loop:** attempt independently → use a hint after about 15-20 minutes if stuck → close the solution → rewrite it yourself. Redo mistakes before adding more questions. Try the HackerRank sample test to learn the interface.

MY NOTES / QUESTIONS TO REVISIT